

LiveWall & MACH8

AI Developer Internship

Candidate: Maria Paula Salazar Agudelo

Project used as proof of work: Fruit Ripeness Classifier (deep learning, computer vision)

Minor: AI & Society

How to use this guide. Part 1 prepares you to talk about your project in the interview and connect it to exactly what LiveWall asked for. Part 2 makes sure you truly understand the foundations, the syntax, and the *why* behind every choice in your own code — so you can answer follow-up questions with confidence. You will not know everything, and that is fine: LiveWall explicitly wants someone who learns by doing and is not afraid to experiment.

Part 1 — The Internship & How to Talk About Your Project

1.1 What LiveWall (MACH8) is actually looking for

The vacancy is written in Dutch. Here is what it really says, decoded:

- **MACH8 is LiveWall's AI hub.** Their mission: help organizations not just *understand* AI, but *apply* it — building solutions that make impact, from tools and automations to innovative products and experiences.
- **What you would do:** help build AI tools, **agents** and prototypes; work with **LLMs and AI APIs**; experiment with new AI tech; support **proof-of-concepts**; think along on technical solutions; collaborate with engineers.
- **Who they want:** a *curious maker* who likes to try things out and grow toward an **AI Engineer** role; a student (HBO/WO), analytical and solution-oriented, **not afraid to make mistakes and experiment**, with **basic programming knowledge** and **experience with side-projects or own builds**.
- **What they offer:** real AI projects, lots of room to learn and experiment, mentorship from experienced engineers, a path to grow into AI Engineer, EUR 500/month, free lunch, 40 hrs/week.

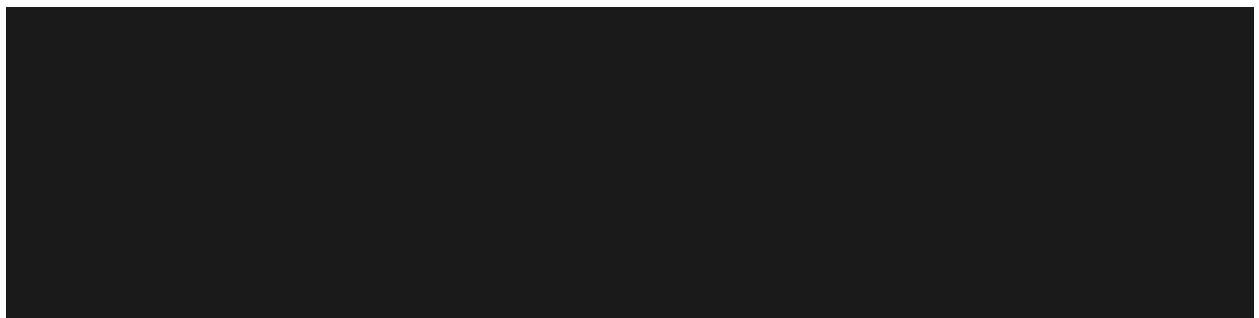
1.2 The honest gap — and how to turn it into your strength

Read the vacancy carefully: their day-to-day leans toward **generative AI** — LLMs, AI APIs, agents. Your Fruit Classifier is **classic computer-vision deep learning**. These are different branches of AI. Do not hide this. Handle it like this:

- **Lead with what matches perfectly:** they want "experience with side-projects or own builds." Your project *is* exactly that — an end-to-end build you made yourself.
- **Show the transferable foundations:** the core skills carry over — understanding models, training, evaluation, preprocessing data, calling a model through an **API**, and shipping something usable. You already wrapped your model in a Flask **API**; that is conceptually the same shape as working with an AI API.
- **Show hunger to grow into LLMs:** say openly that your project taught you the deep-learning foundations, and you are excited to apply that same learning-by-doing approach to LLMs and agents. That sentence aligns word-for-word with their "Over jou."

This honesty is a feature, not a weakness — they literally said "not afraid to make mistakes and experiment."

1.3 Your 30-second pitch



Practice saying it out loud until it feels natural — not memorized word-for-word, but comfortable.

1.4 Mapping your project to what they asked for

What LiveWall asked for	Your evidence (say this)
Experience with side-projects / own builds	The whole project is a self-initiated build — dataset to deployed web app.
Work with AI APIs	You built a REST API with Flask (/api/predict) that receives an image and returns a JSON prediction — same pattern as consuming/serving AI APIs.
Build tools & prototypes / proof-of-concepts	Your web app is a working proof-of-concept: upload a photo, get ripeness in seconds.
Learns by doing, not afraid to experiment	You used data augmentation and transfer learning, iterated, and read the training curves to check for overfitting.
Analytical & solution-oriented	You framed a real problem (choosing good fruit) and chose a lightweight, mobile-friendly model to solve it.
Interest in AI, basic programming	Python, TensorFlow/Keras, NumPy, Flask, SQLite, notebooks — full pipeline in code.

1.5 Likely interview questions & strong answers

Q: Can you walk me through your project?

A: Use the 30-second pitch, then offer to go deeper: "It has three stages — data preparation, model training with transfer learning, and deployment as a Flask API. I can dive into any of them." Let them pick — this shows structure.

Q: Why did you choose this problem?

A: "It is a small everyday problem everyone relates to — you cannot always tell good fruit from a glance. I liked that it was concrete, visual, and something I could realistically ship end-to-end as a learning project."

Q: What was the hardest part / what did you struggle with?

A: Be honest and specific: "Making sure the model generalized instead of just memorizing. I watched the training vs validation curves and used data augmentation and dropout to keep it honest. Also, wiring the trained model into a working API was new to me." (Struggling + fixing it is exactly what they want to hear.)

Q: Your project is computer vision, but we work a lot with LLMs. How do you see that?

A: "Right — my project is classic deep learning, and your focus is generative AI. The branch is different but the foundations transfer: understanding models, prompting/inputs, calling models through APIs, evaluating outputs, and shipping something usable. I built my own API around a model, so I am comfortable with that shape, and I am genuinely excited to learn LLMs and agents hands-on — which is exactly how I taught myself this project."

Q: What would you improve if you had more time?

A: "Three things: finish the TensorFlow Lite conversion so it runs on a phone, collect real user photos to test on messier images, and add proper monitoring of predictions. I already log predictions to a small database as a first step."

Q: How do you learn new technology?

A: "By building. For this project I did not start from a course — I picked a goal, broke it into steps, and learned each piece (transfer learning, augmentation, Flask) as I hit it. That is also why this internship appeals to me: learning by doing with mentorship."

1.6 Smart questions to ask THEM

- "What does a typical proof-of-concept at MACH8 look like — from idea to demo?"
- "Which LLMs or frameworks does the team use most right now (e.g. for agents)?"
- "What would a successful first few months look like for an intern here?"
- "How much do interns get to experiment versus work on client deliverables?"

Asking about *their* tools shows you are already thinking like part of the team.

1.7 Quick prep checklist (the day before)

- Re-read the vacancy and this guide once; say the pitch out loud 3x.
- Have your GitHub repo open and be ready to screen-share the web app and the training script.
- Pick **one** thing you are proud of and **one** thing you struggled with — have both ready.
- Skim Part 2 so the *why* behind each choice is fresh.
- Prepare a one-line answer to "Why LiveWall / MACH8?" — tie it to applying AI for real impact and growing toward AI Engineer.

Part 2 — Understanding Your Project: Foundations, Syntax & Why

2.1 The big picture in one paragraph

Your project is an **image classifier**. You showed a neural network thousands of labeled fruit photos so it could learn the visual patterns of each of 9 categories (fresh / unripe / rotten × apple / banana / orange). Instead of training a network from zero, you took **MobileNetV2** — a network already trained on millions of images — and re-used its visual knowledge (**transfer learning**), adding a small custom "head" that learns *your* 9 classes. After training, the model takes any new photo and outputs 9 probabilities; the highest one is the prediction.

2.2 Core concepts (in plain language)

Supervised learning

You learn from **labeled examples**. Each training image came in a folder named after its class, so the model always knew the "right answer" while learning and adjusted itself to reduce its mistakes.

Classification (vs regression)

The output is a **category** (which of 9), not a number. That single fact drives later choices: **softmax** output and **categorical crossentropy** loss.

Neural network & CNN

A neural network is layers of simple math units ("neurons") that transform the input step by step. For images we use a **Convolutional Neural Network (CNN)**: convolution layers scan the image for patterns — edges first, then textures, then shapes — which is why CNNs are the standard for vision.

Training, epochs, batches

Training = repeatedly showing examples and nudging the model's internal numbers (**weights**) to reduce error. One **epoch** = one full pass over all training images. A **batch** = the small group of images processed at once (yours: 32) before each update.

2.3 Why each decision in your code — the part interviewers probe

Why deep learning / a CNN at all?

Fruit ripeness is a **visual** judgment (color, spots, texture). You cannot write simple if-rules for that. A CNN *learns* those visual features from data, which is exactly what this problem needs.

Why transfer learning?

Training a strong image model from scratch needs millions of images and lots of GPU time — you had neither. Transfer learning re-uses a model that already learned generic vision (edges, shapes, colors) and only teaches it the new, small part (your fruit classes). Result: **far less data, far less time, much higher accuracy**.

Why MobileNetV2 specifically?

- **Lightweight & fast** — designed to run on phones, which fits your "mobile-first" goal.
- **Strong accuracy** for its size, pre-trained on **ImageNet** (~1.4M images).

- Good trade-off: a heavier model (e.g. ResNet) might be slightly more accurate but too big/slow for a phone.

Why freeze the base model? (`base_model.trainable = False`)

Freezing means "do not change MobileNetV2's learned weights." You keep its valuable general vision knowledge and only train your small new head. This prevents wrecking good features with your small dataset, and trains much faster.

Why resize to 224×224?

MobileNetV2 was trained on 224×224 images, so it expects that exact input size. Every image — in training and prediction — must match, which is why the same size appears in both scripts.

Why normalize pixels (`/ 255.0`)?

Pixels come as 0–255. Neural networks train more stably when inputs are small, similar-scale numbers, so you rescale to 0–1. Critical: you must apply the **same** normalization at prediction time, or the model sees "wrong-looking" data.

Why data augmentation (only on training)?

Augmentation makes random variations — rotation, shift, zoom, horizontal flip — so the model sees a fruit from many angles/lighting and learns the *fruit*, not the exact photo. This fights **overfitting**. You do **not** augment the test set, because you want to measure performance on clean, real images.

The custom head, layer by layer

- **GlobalAveragePooling2D** — squashes MobileNetV2's feature maps into a single flat list of numbers, so a normal Dense layer can use them. (Lighter and less overfit-prone than Flatten.)
- **Dense(256, relu)** — a fully-connected layer that learns to combine those features into fruit-specific patterns. **ReLU** is the activation that lets the network model non-linear relationships.
- **Dropout(0.5)** — during training, randomly switches off half the neurons each step. This stops the model from relying on any single neuron and **reduces overfitting**.
- **Dense(9, softmax)** — the final layer: 9 outputs (one per class). **Softmax** turns them into probabilities that add up to 1, so you can read them as "how confident" per class.

Why these training settings?

- **Optimizer = Adam** — a smart, adaptive optimizer that adjusts the learning step automatically; a reliable default.
- **Learning rate = 0.0001 (small)** — because you are fine-tuning on top of a pre-trained model, you take *small careful steps* so you do not overshoot. Too high = unstable; too low = very slow.
- **Loss = categorical_crossentropy** — the standard loss for multi-class, one-label classification with softmax. It measures how far the predicted probabilities are from the true label.
- **Batch size = 32** — a common sweet spot between speed and stable, memory-friendly updates.
- **Epochs = 20** — enough passes for the curves to flatten out (model stops improving) without over-training.

2.4 Syntax rules in YOUR code, explained

The Keras Functional API: `x = Layer(...)(x)`

This pattern looks strange at first. `Dense(256)` creates a layer; the second `(x)` calls it on the previous output `x`. You are chaining layers, each one taking the previous result — like a pipeline.

```
x = base_model.output
x = GlobalAveragePooling2D()(x)      # feed base output into pooling
x = Dense(256, activation='relu')(x)  # then into a dense layer
x = Dropout(0.5)(x)                  # then dropout
outputs = Dense(9, activation='softmax')(x)
model = Model(inputs=base_model.input, outputs=outputs)
```

compile / fit / predict — the three verbs of Keras

```
model.compile(optimizer=Adam(0.0001),      # how it learns
               loss='categorical_crossentropy', # what 'wrong' means
               metrics=['accuracy'])        # what to report

history = model.fit(train_generator,        # TRAIN
                    epochs=20,
                    validation_data=test_generator)

predictions = model.predict(img_array)      # USE the model
```

compile = configure how the model learns. **fit** = actually train it (returns a history of accuracy/loss per epoch, which you plot). **predict** = run a new image through the trained model.

Preparing one image for prediction

```
img = Image.open(path).convert('RGB')  # always 3 color channels
img = img.resize((224, 224))           # match training size
arr = np.array(img).astype('float32') / 255.0  # normalize 0-1
arr = np.expand_dims(arr, axis=0)      # (224,224,3) -> (1,224,224,3)
```

Why expand_dims? The model always expects a *batch* of images, shaped (batch, height, width, channels). For one image you add a batch dimension of 1.

Reading the model's answer

```
predictions = model.predict(arr)        # shape (1, 9)
idx = np.argmax(predictions[0])         # index of the biggest probability
label = class_labels[idx]               # convert index -> fruit name
confidence = predictions[0][idx] * 100  # that probability as a %
```

`np.argmax` returns the *position* of the largest value — i.e. the winning class. You then map that number to a name using the labels file.

```
if __name__ == "__main__":
```

This means "only run this block if the file is executed directly, not when it is imported by another file." It lets `predict.py` be both a runnable script *and* an importable module (your web app imports its functions).

flow_from_directory — how labels are created

You organized images into one folder per class. `flow_from_directory` reads those folder names as the labels automatically, resizes images, batches them, and feeds them to training — no manual labeling code needed.

2.5 The prediction flow, end to end

When someone uploads a photo, this is the exact journey — be ready to narrate it:

- 1. **Receive** the image (file upload or command-line path).

- 2. **Preprocess**: convert to RGB, resize to 224×224, normalize to 0–1, add batch dimension.
- 3. **Infer**: `model.predict()` returns 9 probabilities.
- 4. **Pick the winner** with `np.argmax` and read its confidence.
- 5. **Map** the index to a human label via `class_labels.json`.
- 6. **Return / log** the result (JSON from the API; a row in `predictions.db`).

2.6 How your web app works (this connects to "AI APIs")

Your `webapp/app.py` uses **Flask** to expose your model over HTTP. This is exactly the "work with AI APIs" idea, so know it well:

- GET / — serves the HTML page where a user picks a photo.
- POST /api/predict — receives the image, runs the model, returns JSON: predicted label, confidence, and all 9 probabilities.
- GET /api/health — a simple check that the server and model loaded correctly.

CORS is enabled so a separate front-end (React, a mobile app) could call this API from another origin. *Note for honesty*: the web app currently loads a `.tflite` model that is not generated yet — mention this as known future work if asked (see 2.10).

2.7 Metrics: what the numbers mean

- **Accuracy** — the share of images predicted correctly. Easy to explain, but not the whole story.
- **Loss** — a continuous measure of how wrong the probabilities are; the optimizer minimizes it. Should *decrease* over epochs.
- **Validation accuracy/loss** — measured on images the model did *not* train on. This is the honest signal of real performance.
- **Overfitting** — when training accuracy is high but validation accuracy lags: the model memorized instead of learning. Your defenses: data augmentation, dropout, and a frozen base. You explicitly watch the gap between the two curves.

2.8 Quick glossary (skim before the interview)

Term	Plain meaning
Transfer learning	Re-using a pre-trained model and adapting it to a new task.
ImageNet	A huge labeled image dataset used to pre-train MobileNetV2.
Weights	The internal numbers the model adjusts while learning.
Epoch	One full pass over all training data.
Batch	Small group of samples processed before each weight update (yours: 32).
ReLU	Activation function; lets the network learn non-linear patterns.
Softmax	Turns final outputs into probabilities that sum to 1.
Dropout	Randomly disables neurons in training to reduce overfitting.
Crossentropy	Loss function for classification; penalizes confident wrong answers.
Adam	Adaptive optimizer; a solid default for training.

Term	Plain meaning
Inference	Using a trained model to predict on new data.
Overfitting	Memorizing training data; poor on new data.

2.9 Likely technical questions & crisp answers

Q: What is transfer learning and why use it here?

A: Re-using a model pre-trained on millions of images and adapting it to my 9 classes. I used it because I did not have huge data or GPU time, and it gives much higher accuracy faster.

Q: Why softmax on the last layer?

A: It converts the 9 raw outputs into probabilities that sum to 1, so I can read confidence per class and pick the highest with argmax.

Q: What is dropout doing?

A: During training it randomly turns off neurons so the model cannot lean on any single one. It reduces overfitting and helps it generalize.

Q: How do you know it is not just memorizing?

A: I compare training vs validation curves. If validation tracks training closely, it is generalizing; a big gap would mean overfitting. I also used augmentation and dropout as defenses.

Q: Why normalize the images?

A: Networks train more stably with small, consistent input values, so I scale pixels from 0-255 to 0-1 — and I apply the exact same step at prediction time.

Q: What would break if you fed a 500x500 image straight in?

A: The model expects 224x224, so I must resize first; otherwise the input shape would not match and it would error or give garbage.

2.10 Honest limitations (saying these shows maturity)

- **TensorFlow Lite not finished** — the mobile model is planned; the script exists but I have not generated the `.tflite` yet, so the web app should be pointed at the `.keras` model until then.
- **Tested on dataset images** — real phone photos (messy backgrounds, odd lighting) would be the true test; collecting those is next.
- **Only 3 fruits, 3 stages** — the scope is deliberately small to ship something working.
- **Ripeness is subjective** — the boundary between "fresh" and "unripe" can be fuzzy even for people, so some confusion is expected.

Knowing the limits of your own work is exactly the analytical, honest mindset LiveWall asked for. Good luck, Maria Paula — you have got this.